

# MIDAS GUI — User Guide

---

## MIDAS GUI — User Documentation

---

**Version:** 1.0.0 **Application:** `midas-gui` (or `python -m midas_gui`) **Backends:** `midas_calibrate_v2`, `midas_integrate_v2`, `midas_calibrate`, `midas_hkls`, `midas_distortion` **Last updated:** 2026-07-07

**Maintenance:** keep this document in sync with the code — whenever the workflow or a tab's controls change, update the relevant section here in the same change.

**Tab status:** Tabs **0–4** (Data Viewer, Mask Builder, Calibrate, Calib. Refinement, Batch Integrate) are **verified** and ready to use. Tabs **5–8** (Corrections & Physics, PDF Analysis, Texture, Results & Export) are a **work in progress** and will be updated in the coming weeks.

---

## Table of Contents

---

1. [Overview and Architecture](#)
  2. [Getting Started](#)
  3. [Tab 0 — Data Viewer](#)
  4. [Tab 1 — Mask Builder](#)
  5. [Tab 2 — Calibrate](#)
  6. [Tab 3 — Calibration Refinement](#)
  7. [Tab 4 — Batch Integrate](#)
  8. [Tab 5 — Corrections & Physics](#)
  9. [Tab 6 — PDF Analysis](#)
  10. [Tab 7 — Texture / Pole Figure](#)
  11. [Tab 8 — Results & Export](#)
  12. [Common UI Conventions](#)
  13. [Packaging, Deployment & Diagnostics](#)
- 

## 1. Overview and Architecture

---

The MIDAS GUI is a 9-tab PyQt5 desktop application that exposes the scientific capability of the `midas_calibrate_v2` and `midas_integrate_v2` packages through a structured workflow. The intended order of use is:

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| Data Viewer (inspect) → Mask Builder → Calibrate → [Refine] → Batch Integrate |           |
|                                                                               | →         |
| Corrections preview                                                           |           |
|                                                                               | → PDF     |
| Analysis                                                                      |           |
|                                                                               | → Texture |
| / Pole Figure                                                                 |           |
|                                                                               | → Results |
| & Export                                                                      |           |

**Cross-tab shared state.** When Tab 2 (Calibrate) produces a result it is automatically propagated to all downstream tabs. When Tab 1 (Mask Builder) computes a mask it is sent to the consuming tabs. Tab 3 (Refinement), if applied, re-broadcasts the refined geometry. No manual copying is needed.

**All heavy computation runs off the GUI thread.** Every operation that touches a MIDAS package (calibration, integration, mask training, PDF transform, gain training, field averaging, drift fitting) runs in a background `QThread` worker; the GUI stays responsive and log lines stream in real time.

### Package layout.

|                       |                                                   |
|-----------------------|---------------------------------------------------|
| midas_gui/            |                                                   |
| ├ app.py              | MainWindow, dark theme, crash diagnostics, main() |
| ├ __main__.py         | enables `python -m midas_gui`                     |
| ├ style.py            | Dioplas-inspired QSS theme + layout helpers       |
| ├ constants.py        | calibrants, colormaps, dtype sentinels, default   |
| paths                 |                                                   |
| ├ helpers.py          | image IO, geometry parsing, spec building, no-    |
| scroll widgets        |                                                   |
| ├ widgets.py          | ImageViewer / PickableImageViewer / ProfileViewer |
| / FieldSelector / ... |                                                   |
| ├ workers.py          | all QThread workers + the integration core        |
| ├ calib.py            | calibration-pipeline dispatch                     |
| ├ dialogs.py          | save dialogs                                      |
| └ tab_*.py            | one module per tab                                |

**Layout pattern.** The four analysis tabs — **0 Data Viewer, 2 Calibrate, 3 Calib. Refinement, 4 Batch Integrate** — use a **three-panel layout**:

[ Data Loader | Parameters | Display / results ]

- **Data Loader (left).** A shared `DataLoaderPanel` for selecting the five inputs — **Data, Dark, Bright, Background, Mask** — each as a single file, a folder, or an HDF5 dataset (a container dropdown appears for HDF5). Frame controls live here too (Tab 0 navigator, Tab 2/3 frame index, Tab 4 frame range + stride).
- **Parameters (middle).** The tab's analysis controls.

- **Display / results (right).** Image viewer, plots, and/or a log panel.

The three panels are separated by **draggable splitter handles** — drag the `Data Loader | Parameters` and `Parameters | Display` boundaries to rebalance widths to taste. Each panel has a minimum width; drag past it and the panel shows a scrollbar rather than clipping its controls.

The remaining tabs (1, 5–8) keep the classic two-panel layout.

**Corrections & mask (all four analysis tabs).** Dark is averaged then subtracted; Bright is averaged then flat-field **divided** or **subtracted** (your choice); Background is averaged then subtracted; every enabled Mask source (files/folders plus the auto-added "Tab 1 mask") is **unioned** into one composite mask that zeroes/ignores those pixels. Correction order: `(img - dark) → bright → - background → clip ≥ 0`.

---

## 2. Getting Started

---

`midas-gui` is **not on PyPI** — install it from source with conda.

```
git clone https://github.com/d-beniwal/MIDAS_GUI.git
cd MIDAS_GUI
conda env create -f environment.yml      # creates the 'midas-gui' env and
                                         installs the GUI
conda activate midas-gui
midas-gui                               # launch (equivalently: python -m
                                         midas_gui)
```

The GUI drives the MIDAS analysis backends (`midas-calibrate-v2`, `midas-integrate-v2`, `midas-calibrate`, `midas-hkls`, `midas-distortion`), which are also not on PyPI — point the `pip:` section of `environment.yml` at your MIDAS source before creating the environment (see the comments in that file).

### Test data

Synthetic test data lives in the repo-root `test_data/` folder (git-ignored):

- `calibrant_ceria.tif / .h5` —  $\text{CeO}_2$  calibrant (Eiger2 500K, 1028×512, 75  $\mu\text{m}$  pixels)
- `nickel_tifs/` — 10-frame Ni scan, lattice expanding +0.1 %/frame
- `nickel_stack.h5` — the same scan as one HDF5 stack (`exchange/data`)

Default geometry:  $\lambda = 0.39 \text{ \AA}$ , Lsd = 121 mm, pixel = 75  $\mu\text{m}$ , BC = (10, 10) px. Every tab's default file paths point at this data and **auto-load on startup when present**, so the app is usable immediately after checkout.

---

### 3. Tab 0 — Data Viewer

**Purpose:** inspect detector frames and do a quick geometry-free radial integration. Produces no shared state for other tabs.

#### Data Loader panel (left)

Data, Dark, Bright, Background and Mask are all selected in the shared **Data Loader panel** (see §1): each accepts a file / folder / HDF5-dataset. The **Data** card here holds the **frame navigator** (slider, spin box, ◀/▶); **zoom/pan is preserved** as you step through frames (the view only auto-frames on a fresh load). Dark/bright/background and the composite mask are applied to the displayed image and to the radial integration.

#### Projection card

Collapse a stack to one image: **Max** (hot-pixel hunting), **Sum** (long-exposure equivalent), or **Average** (noise reduction), along a chosen axis (0 = across frames).

#### Calibration card (optional)

Load geometry from a **calibration .json**, a **MIDAS paramstest.txt**, or a **pyFAI .poni** (auto-detected). It fills BC, Lsd, pixel size and wavelength, unchecks "Beam centre = image centre", and refreshes the overlay and radial plot.

#### Intensity range card (radial integration)

| Field                       | Description                                                                                                                                |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| Exclude out-of-range pixels | When on, pixels $\leq$ min or $>$ max are drawn as a red overlay and excluded from the radial integration (removes gaps / hot / overflow). |
| pixel $\leq$ / pixel $>$    | Lower / upper bounds. On load, the upper bound auto-fills to <b>max(99.99th percentile, 100000)</b> .                                      |

#### Ring simulation card

Overlays simulated Debye-Scherrer rings to check geometry.

- **Material** dropdown (CeO<sub>2</sub>, LaB<sub>6</sub>, Si, Al<sub>2</sub>O<sub>3</sub>, Cu, Ni, FCC- $\gamma$ Fe, BCC- $\alpha$ Fe, Au, Ag, Pt, W, Ti) or **Custom**.
- Lattice entry is compact: **a**, **b**, **c** on one row and  **$\alpha$** ,  **$\beta$** ,  **$\gamma$**  on the next, plus **SG #**. A **Cubic (a=b=c,  $\alpha=\beta=\gamma=90^\circ$ )** checkbox lets you enter only **a** for cubic crystals (b, c mirror a; angles fixed at 90°). Lattice fields are editable only for Custom.
- Geometry:  $\lambda$ , max 2 $\theta$ , Lsd, pixel size, and beam centre (auto = image centre, or manual BC\_y/BC\_z). The  $\lambda$  label is clickable (underlined) — click it for a menu of common K-edge foils (Pr, Sm, Yb, Lu, Hf, Ta, W, Re, Pt, Au, Pb, Bi) that sets  $\lambda$  to that element's K absorption-edge wavelength. The same clickable- $\lambda$  menu is on the Calibrate and PDF tabs.
- **Show rings / Labels** toggles; **Simulate rings** draws the overlay + hkl table.
- → **Send geometry to Calibrate** copies  $\lambda$ , pixel size, Lsd and beam centre into the Calibrate tab's detector + seed fields (the Calibrate tab has a matching ← **Data Viewer**

button that pulls the same values).

## Beam-centre picking (on the image)

The image viewer has **Pick BC** (single click sets the beam centre) and **Pick Ring** (click  $\geq 3$  points on a ring; a circle fit estimates the beam centre). Either updates BC\_y/BC\_z and re-runs the overlay + radial integration.

## Radial integration plot (bottom-right)

Below the image is a live **azimuthal average about the beam centre** (`R bin`, `Integrate`, and an `Auto` toggle that recomputes on frame/BC/mask change). It is a simple geometry-free radial average (flat detector, no tilt/distortion). **Clicking a radius on the plot draws the matching ring (magenta) on the image.** Axis units switch between R (px) /  $2\theta$  / Q.

---

## 4. Tab 1 — Mask Builder

---

**Purpose:** identify and exclude bad pixels. The final mask is a uint8 array (1 = bad) broadcast to Tabs 2, 3, 4, 5. Browsing an image or a mask loads it immediately.

### Section 1 · Threshold mask (always applied)

`pixel ≤ lower | pixel > upper`. The upper bound auto-fills from the data type on load (e.g. 1,048,575 for uint20 Eiger, 4,294,967,295 for uint32).

### Section 2 · Statistical auto-mask

Two **independently selectable** methods — enable either or both:

- **Spatial outlier** — robust local-anomaly detection (5×5 median residual → 15×15 local MAD → Z-score → hot/dead/saturated gates). Uses a **temporal median** over the stack folder when provided (cleaner reference), else the single frame. Controls: `K_σ` (6.0), `Hot` (1.5), `Dead` (0.5).
- **Temporal constancy** — flags frozen pixels whose frame-to-frame std is below `Frozen × Q75(std)` (0.05); needs a stack of  $\geq 2$  frames.

A shared **stack source + stride** feeds both (the temporal median for spatial, the frame stack for temporal constancy). The stack can be a **folder / \*.tif glob**, or a **single HDF5 file whose 3-D dataset is a time sequence of images** — for an `.h5` a **Dataset** selector appears (auto-populated, 3-D datasets preferred). *All  $\sigma$  /  $K_\sigma$  /  $n_\sigma$  fields in this tab accept any value — there is no upper limit.*

### Section 3 · Spatial spike rejection

Laplacian high-pass single-pixel-spike detector; `n_σ` threshold (5.0).

## Section 3b · Cosmic-ray rejection (temporal)

Per-pixel temporal  $\sigma$ -clip across a  $\geq 3$ -frame stack; anomalies OR'd across frames. `n_σ` (5.0).

## Section 4 · Calibration-based masks (need a Tab 2 result)

- **Azimuthal  $\sigma$ -clip** — flags (R,  $\eta$ ) cells deviating from the azimuthal mean.
- **Learnable mask** — differentiable per-pixel mask trained against an  $\eta$ -uniformity loss (`steps`, `lr`, `sparsity`).

### Draw mask tools

Interactive rectangle / oval / circle / polygon / annulus / point tools, live-draggable; **Apply shapes** → **mask** rasterises them (OR'd with the computed mask). **Clear shapes** removes them.

### Save / Load

Save the combined mask as TIFF (0 = good, 1 = bad); loading a TIFF applies immediately.

---

## 5. Tab 2 — Calibrate

---

**Purpose:** determine detector geometry (Lsd, BC, tilts, distortion, wavelength) from a calibrant pattern.

### Data Loader panel (left)

The calibrant frame (Data + Frame index), Dark, Bright, Background and Mask are selected in the shared **Data Loader panel** (see §1). Each Dark/Bright/Background field is averaged over a chosen index range; Bright offers **Flat-field divide** or **Subtract**; all mask sources (plus the auto-added Tab 1 mask) are unioned. Dark is passed to the calibration pipeline; bright/background are applied to the calibrant before calibration and post-calibration integration.

### Detector, seed & Load calibration file

The **Detector & Calibrant** card sets  $\lambda$ , pixel size(s) and detector transforms; the **Initial seed** card sets the LM starting point (BC\_y, BC\_z, Lsd). **Load calibration file...** (top of the Detector card) reads a MIDAS **paramtest** `.txt`, a calibration `.json`, or a pyFAI `.poni` (auto-detected) and fills  $\lambda$ , pixel size, and the seed BC + Lsd — a fast way to start from a previous calibration or a known geometry. (The synthetic test data ships `test_data/calibration_synthetic.{json,txt,poni}` as a ready example.) ← **Data Viewer** (next to *Load calibration file...*) pulls  $\lambda$ , pixel size, Lsd and beam centre straight from the Data Viewer tab into the same fields.

## Pipeline selector

One-shot (default) · First-time · Four-stage · Bayesian (Laplace  $\sigma$ ) · Joint-cake. *For trustworthy tilt/strain, prefer Four-stage or First-time — One-shot/Bayesian can report a spurious self-compensated tilt on weakly-tilted data.*

## Refine flags

Which parameters vary: Lsd, BC, ty, tz, tx, Wavelength, Distortion (15 coeffs), plus a "Residual map" build toggle. Advanced (E-M / LM iters, device, output dir) and Multi-panel detector groups are collapsible.

## Live threshold slider

Zeroes calibration-image pixels below the slider value (background suppression for ring-finding); the preview updates instantly.

## Pick BC / Pick Ring

Click the image to seed the beam centre (single click) or fit a ring ( $\geq 3$  clicks).

## Run / Abort

**Run Calibration** launches the worker; **Abort** terminates it and frees the slot so you can immediately start a new run (the calibration is one uninterruptible library call, so abort hard-terminates the worker thread rather than waiting).

## Results (right panel, bottom tabs)

Radial Profile (with ring markers and an **Azim. avg** selector — see Tab 4), Ring Residuals bar chart, Results (Lsd/BC/strain + distortion table), and Log. Integration runs automatically after calibration.

## Export

**Save calibration.json** and **Save paramstest.txt** (standalone or from a template).

---

## 6. Tab 3 — Calibration Refinement

---

**Purpose:** optional post-calibration geometry refinement against an  **$\eta$ -uniformity** criterion (rings should be azimuthally uniform). Uses a **derivative-free Nelder-Mead** optimiser (the differentiable integrator returns NaN geometry gradients at the beam-centre singularity on this build).

The sample frame and Dark/Bright/Background/Mask are selected in the shared **Data Loader panel** (see §1) — the refinement runs on the corrected image. Middle-panel controls: calibration source (Tab 2 result, or start-from/continue radios), parameters to refine

(BC\_y, BC\_z, Lsd, ty, tz), optimiser + iterations. **Run Refinement** shows a live loss curve; **Apply** broadcasts the refined geometry downstream. If Apply is never clicked, the Tab 2 result flows through unchanged.

---

## 7. Tab 4 — Batch Integrate

---

**Purpose:** integrate a stack of frames into 1-D profiles using the calibrated geometry and optional corrections.

### Data Loader panel (left)

The streaming **Data** source (folder/glob or HDF5 dataset) with **frame range + stride**, plus Dark/Bright/Background and Mask, are selected in the shared **Data Loader panel** (see §1). Dark/bright/background are applied per frame; all mask sources are unioned.

### Calibration source (middle)

- **From Tab 2** — the calibration (or refined) result.
- **From file** — a **calibration .json**, **MIDAS paramstest.txt**, or **pyFAI .poni** (auto-detected; both GUI and MIDAS-pipeline json key styles supported). Entering a path auto-selects this option.

### Integration

| Field                          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Kernel                         | Hard (fastest) · Subpixel K=2 · Subpixel K=4 · Polygon (exact).                                                                                                                                                                                                                                                                                                                                                                                       |
| R bin (px)<br>/ $\eta$ bin (°) | Radial and azimuthal bin sizes.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| <b>Azim. avg</b>               | How the ( $\eta$ , R) cake becomes a 1-D profile: <b>Pixel-weighted</b> (default) $\Sigma(\text{mean} \cdot \text{count}) / \Sigma(\text{count})$ — independent of $\eta$ -bin size and robust to partial azimuthal coverage / <b>off-detector beam centres</b> ; or <b><math>\eta</math>-bin mean (legacy)</b> — the unweighted mean of per- $\eta$ -bin means, which can distort with a coarse $\eta$ bin when the beam centre is off the detector. |
| Per-bin variance ( $\sigma$ )  | Error model poisson / azimuthal / hybrid (ignored when corrections are on $\rightarrow \sigma = \sqrt{l}$ ).                                                                                                                                                                                                                                                                                                                                          |
| Q-uniform bins                 | Integrate in R then rebin onto a uniform-Q grid (Qmin, Qmax, $\Delta Q$ ).                                                                                                                                                                                                                                                                                                                                                                            |

### Physics corrections

Polarization and solid-angle (pixel-domain, via `integrate_with_corrections`).

### Monitor normalisation

Divide each processed frame's profile/ $\sigma$  by a per-frame scalar from a text file (one value per *processed* frame).



## Drift correction (long scans)

Per-frame geometry interpolated from an anchor JSON ( `{frame_idx: {Lsd, BC_y, BC_z}}` ) with spline/linear/constant parametrization; **Fit trajectory** builds it. Each frame is then integrated with its own geometry.

## Run / Abort / Clear

**Start Integration / Abort.** Abort first asks the worker to stop cleanly between frames (keeping frames already written); if it does not stop promptly it force-terminates and frees the slot. Completed frames' output files are preserved. **Clear results** removes the profiles/plots computed **this session** for the current data (waterfall, stacked profiles, and the integrated-frame tracking) so a fresh integration can start — it stops any active monitor but does **not** delete raw data or any files already written to disk.

## Live folder monitoring (MONITOR)

The **MONITOR** button at the bottom of the left Data Loader panel (folder/glob sources only) starts a live watch: while active it turns **green** and the GUI polls the data folder for **new** TIFF frames, integrating each one **as it appears** and adding it to the display. It does **not** re-run the whole batch — it reuses the already-built **detector map** (the binning geometry / pixel-count cakes; reused from a prior *Start Integration* run when the calibration, kernel, bins, mask and folder are unchanged, or built once on first use) and integrates **only the new files** (tracked by frame id, so frames already shown are skipped). New frames honour the current kernel, corrections, Dark/Bright/Background, mask and Q-uniform settings, and are saved to the output folder when a 1-D format is selected. Click MONITOR again to stop; starting a fresh *Start Integration* also stops it. (Distinct from **Monitor normalisation** above, which divides profiles by a scalar file.)

## Output formats

CSV (R,I, $\sigma$ ) · XYE (2 $\theta$ ) · FXYE (centideg) · DAT (Q) · HDF5 (full stack) · 2D-CSV ( $\eta \times R$  cake). Right panel: live **Waterfall** and **Stacked profiles** — both have an **x** selector to show the axis in **R (px) / 2 $\theta$  (°) / Q (Å<sup>-1</sup>)** (converted from the run's calibration).

The **Stacked profiles** view is a publication-quality plot: each curve tags its **source file name inline, just below the curve at its left edge** (toggle with the *Labels* checkbox; an optional corner *Legend* is also available). A **theme** selector switches between two saved presets:

- **White (publication)** (*default*) — white background, **point + line** markers, a colour-blind-friendly categorical palette (matplotlib *tab10*), a boxed frame and a light grid — ready to drop into a paper.
- **Dark** — the classic on-screen look (dark background, line-only, vivid golden-angle colours).

An **x** selector plots the axis in **R (px)**, **2 $\theta$  (°)** or **Q (Å<sup>-1</sup>)** (converted from the run's calibration). The **Grid** checkbox (off by default) toggles the horizontal + vertical grid. The *spacing* box shifts successive curves vertically (0 = overlay). Top-right controls adjust the **line width** ( –

line + ), **symbol size** ( - sym + , for the point+line markers — also turns markers on if a line-only theme is active) and **label font size** ( - font + ).

---

## 8. Tab 5 — Corrections & Physics (*work in progress*)

---

Preview physics corrections on a single frame (auto-loads on browse). Pixel-domain: polarization, solid angle, empty subtraction. Profile-domain: cylindrical absorption ( $\mu R$ ), Compton subtraction (composition:fraction). **Compute corrected profile** overlays corrected vs uncorrected and shows the correction factor vs  $2\theta$ .

Also hosts **per-pixel gain training (LearnableGain)**: from a clean reference frame and a drifted frame, learn a spatial gain map  $g_i = 1 + \text{scale} \cdot r_i$  by minimising  $\text{MSE}(\text{profile}) + \text{unity} \cdot \sum (g_i - 1)^2 + \text{smooth} \cdot \text{TV}(g)$  ; save as NPZ and apply with  $\text{corrected} = \text{raw} / \text{gain\_map}$  .

---

## 9. Tab 6 — PDF Analysis (*work in progress*)

---

Polyatomic **total-scattering** reduction:  $I(Q) \rightarrow$  Faber-Ziman structure function  $S(Q) \rightarrow$  pair-distribution  $G(r)$ , powered by the `midas_pdf` backend (real composition-weighted normalization, Compton subtraction, end-to-end  $\sigma$  propagation, and optional differentiable scale/background refinement). This replaces the earlier monoatomic, composition-free  $F(Q) = Q \cdot (I/bg - 1)$  approximation.

### Background — what $I(Q)$ , $S(Q)$ and $G(r)$ mean

PDF analysis is three transforms that move from *what the detector measured* to *where the atoms are*. They are exactly the three stacked plots (top  $\rightarrow$  bottom).

- **$I(Q)$  — measured scattered intensity.** Intensity vs. momentum transfer  $Q = 4\pi \cdot \sin(\theta) / \lambda$  ( $\text{\AA}^{-1}$ ), a wavelength-independent rescaling of the angle  $2\theta$ ; high  $Q$  = wide angle = fine spatial detail. Obtained by azimuthally integrating the detector rings to a 1-D curve.  $I(Q)$  is **not** pure structure — it mixes the interference (coherent) signal with big smooth backgrounds: per-atom self-scattering (form factors  $\langle f^2 \rangle$ ) and inelastic **Compton** scattering.
- **$S(Q)$  — structure function.**  $I(Q)$  with the self-scattering and Compton removed and normalized away (the **Faber-Ziman** step), leaving only interference:  $S(Q) = [I_{\text{coh}} - (\langle f^2 \rangle - \langle f \rangle^2)] / \langle f \rangle^2$ , where  $\langle f \rangle$ ,  $\langle f^2 \rangle$  are composition-weighted atomic form factors (hence the **Composition** input).  $S(Q)$  oscillates about **1** and  $\rightarrow 1$  at high  $Q$  where correlations wash out (the dashed guide line).  $F(Q) = Q \cdot [S(Q) - 1]$  is the same thing re-weighted by  $Q$  to emphasize the structurally rich high- $Q$  oscillations.
- **$G(r)$  — reduced pair distribution function.** The real-space answer: a histogram of interatomic distances, via a sine Fourier transform  $G(r) = (2/\pi) \cdot \int Q \cdot [S(Q) - 1] \cdot \sin(Qr) \, dQ$ . A **peak at  $r$**  means many atom pairs are separated by that distance; the **first peak is the nearest-neighbour bond** ( $\text{Ni} \approx 2.49 \text{ \AA}$ ,  $\text{CeO}_2$   $\text{Ce-O} \approx 2.34 \text{ \AA}$ ), later peaks are further coordination shells. Below the first bond  $G(r) = -4\pi \rho_0 r$  (nothing can

be closer) — a constraint that needs the number density  $\rho_0$ . Because it uses *total* scattering (Bragg **and** diffuse), the PDF captures local structure even in disordered / nanoscale / amorphous materials, unlike a Bragg-only Rietveld fit.

```

detector image / I(Q) file
  | azimuthal integration
  ▼
  I(Q) raw intensity = interference + form factors + Compton
(top plot)
  | subtract Compton, subtract Laue ( $\langle f^2 \rangle - \langle f \rangle^2$ ), divide by  $\langle f \rangle^2$ 
(needs composition)
  ▼
  S(Q) pure interference, oscillates about 1
(middle plot)
  | Fourier sine transform over [Qmin, Qmax] with a window
(Lorch)
  ▼
  G(r) interatomic distances; peaks = coordination shells
(bottom plot)

```

The  $\pm 1\sigma$  band on G(r) is the counting uncertainty  $\sigma$  propagated from I(Q) through every step, so real peaks can be told from noise. The **G/g/T/R** family (Output selector) is the same information re-weighted: **G** oscillates about 0 (refinement standard); **g**  $\rightarrow 1$  at large r; **T** is the total correlation; **R** is the radial distribution whose *peak area = coordination number*. g/T/R all need  $\rho_0$ .

**I(Q) source** (choose one):

- **Integrate detector frame** — a calibrated frame is integrated (hard/polygon binning, Poisson  $\sigma$ ) and mapped to Q, exactly as before. Uses the Tab 2 calibration ( $\lambda$ , geometry) and any Tab 1 mask.
- **Load I(Q) file** — a pre-integrated 2- or 3-column Q, I,  $\sigma$  text/CSV (comment/header lines tolerated). The tab **opens on this source by default**, pointed at real data in `test_data/pdf_real/` ( `iq_Nickel.csv` ; also `CeO2`, IPA, Kapton at  $\lambda$  0.1839) — just press **Compute G(r)** for the Ni PDF (first peak  $\sim 2.49$  Å). Synthetic known-answer examples are in `test_data/pdf_synth/` ( `nickel_iq.csv` , `ceo2_iq.csv` ). See each folder's `README.md` for per-sample composition /  $\rho_0$  / Q-range settings.

**Calibration.** The geometry/wavelength comes from Tab 2 automatically, or use **Load calibration file...** to read a MIDAS `paramtest.txt` , a `.json` , or a pyFAI `.poni` (auto-detected). This is required for *Integrate detector frame* and also sets  $\lambda$  used by *Load I(Q) file* mode.

**Sample.** Composition (e.g. `Ni` or `C:3,H:8,O:1` ; ions like `Ni2+` allowed), number density  $\rho_0$  (atoms/Å<sup>3</sup>; needed for refinement and for g/T/R), wavelength  $\lambda$  (auto-filled from calibration), and a **Compton subtraction** toggle.

**Normalization.** Optional **Refine scale + background** (L-BFGS) — reveals a background polynomial degree (0–3), a low-r cutoff `r_min` (Å), and an iteration count. Refinement requires  $\rho_0 > 0$ . It fits scale and a smooth  $b(Q)$  against two model-free constraints:  $\langle S \rangle \rightarrow 1$  and  $G(r) = -4\pi\rho_0 r$  below `r_min`.

**Output.** Bottom-plot convention family — **G(r)** (reduced PDF), **g(r)** (pair distribution), **T(r)** (total correlation), or **R(r)** (radial distribution, whose peak integral is the coordination number);  $g/T/R$  need  $\rho_0$ . Middle plot toggles between **S(Q)** (with the  $S=1$  guide) and the true reduced structure function **F(Q)=Q(S–1)**.

Right panel:  $I(Q)$  (+ refined background overlay),  $S(Q)/F(Q)$ , and the selected  $G/g/T/R$  with a shaded  $\pm 1\sigma$  band. **Save G(r)** writes a three-column `r, G(r),  $\sigma$`  file (diffpy-CMI / PDFgui / RMCProfile compatible); **Save S(Q)** writes `Q, S(Q)`.

## Functions behind the tab

The tab is a thin UI over a background worker and the `midas_pdf` backend.

### UI — PDFTab (`midas_gui/tab_pdf.py`)

| Method                                                                         | Role                                                                                                          |
|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <code>_build_ui</code>                                                         | Builds the $I(Q)$ -source / Sample / Normalization / Output groups and the three stacked plots.               |
| <code>set_calibration(result, source)</code>                                   | Receives a Tab-2 result (or a loaded geometry) → sets $\lambda$ and enables Compute.                          |
| <code>_load_calib_file</code>                                                  | Loads a <code>paramtest / .json / .poni</code> → builds a calibration result → <code>set_calibration</code> . |
| <code>_load_img</code>                                                         | Loads a detector frame for <i>Integrate detector frame</i> mode.                                              |
| <code>_run</code>                                                              | Validates inputs, assembles the <code>cfg</code> dict, and starts a <code>PDFWorker</code> .                  |
| <code>_on_done</code> / <code>_redraw_mid</code> / <code>_redraw_bottom</code> | Draw $I(Q)+bg$ , the $S(Q) \leftrightarrow F(Q)$ toggle, and the $G/g/T/R$ curve with its $\pm 1\sigma$ band. |
| <code>_save_gr_file</code> / <code>_save_sq_file</code>                        | Export <code>r, G, <math>\sigma</math></code> and <code>Q, S</code> .                                         |

### Worker — PDFWorker (`midas_gui/workers.py`) runs off the GUI thread:

| Function                 | Role                                                                                                                                                                                               |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_acquire_iq</code> | Produces <code>(q, I, <math>\sigma</math>)</code> — either by integrating the frame (image mode) or by reading a file ( <code>_load_iq_file</code> , tolerant of comma/space and 2- or 3-columns). |
| <code>run</code>         | Trims to <code>[Qmin, Qmax]</code> , builds <code>Composition</code> , calls the reduction (plain or refine), computes <code>F(Q)</code> and the $G/g/T/R$ family, emits the result dict.          |

Image-mode integration reuses the shared core `_build_spec`, `build_geom`, `integrate_frame`, `axis_conversions` (same path as the other tabs); geometry-file parsing uses `geometry_fields_from_file` / `result_ns_from_geometry_file` (`midas_gui/helpers.py`).

**Reduction** — `midas_pdf` via `midas_gui/pdf_backend.py` (re-exported symbols):

| Function                                                                       | Does                                                                                                                                                               | Maps to plot                    |
|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|
| <code>Composition(fractions, number_density)</code>                            | Composition layer: $\langle f \rangle$ , $\langle f^2 \rangle$ form-factor averages, Laue term $\langle f^2 \rangle - \langle f \rangle^2$ , and per-atom Compton. | —                               |
| <code>faber_ziman_S(I, q, comp, ...)</code>                                    | $I(Q) \rightarrow S(Q)$ with $\sigma$ (subtract Compton/Laue, divide by $\langle f \rangle^2$ ).                                                                   | $S(Q)$                          |
| <code>structure_function_F(q, S)</code>                                        | $F(Q) = Q \cdot (S - 1)$ .                                                                                                                                         | $F(Q)$                          |
| <code>fourier_sine_transform(q, S, r, window)</code>                           | $S(Q) \rightarrow G(r)$ with $\sigma$ (Lorch / none window).                                                                                                       | $G(r)$                          |
| <code>i_of_q_to_Gr(q, I, comp, r, ...)</code>                                  | End-to-end $I(Q) \rightarrow G(r)$ (composes the two above); the default Compute path.                                                                             | $S(Q) + G(r)$                   |
| <code>refine_normalization(q, I, comp, r, ...)</code>                          | L-BFGS fit of scale + background polynomial against high-Q $\langle S \rangle \rightarrow 1$ and low-r $G = -4\pi p_0 r$ .                                         | refined $S(Q)/G(r) + \text{bg}$ |
| <code>pair_distribution_g / total_correlation_T / radial_distribution_R</code> | Convention family from $G(r) + p_0$ .                                                                                                                              | $g/T/R$                         |

**Packaging note.** `midas_pdf` is currently **vendored** in `midas_gui/_vendor/` and loaded through `midas_gui/pdf_backend.py`, which also installs a small `midas_hkls.absorption` compatibility shim for `midas-hkls < 0.5.0`. Once `midas-pdf` is published and `midas-hkls >= 0.5.0` is available the vendored copy and the shim are removed.

## 10. Tab 7 — Texture / Pole Figure (*work in progress*)

Per-ring azimuthal analysis for preferred orientation. Controls: calibration source, sample frame,  $R/\eta$  bins, ring index,  $\chi$  (tilt) /  $\phi$  (rotation). **Compute Pole Figure** integrates to a cake and extracts  $I(\eta)$  at the selected ring; the right panel shows the stereographic pole figure and the raw  $I(\eta)$ . **Save pole figure (.pol)** exports POPLA format.

## 11. Tab 8 — Results & Export (*work in progress*)

Session summary + one-click export. Checkboxes select which products (calibration.json, paramtest.txt, mask.tif, integrated profiles,  $G(r)$ , pole figures, session log) to copy to an output directory. A provenance block (package versions, geometry hash, mask fraction, correction flags) can be copied to the clipboard for a Methods section.

## 12. Common UI Conventions

---

- **Browse = load.** Selecting a file/folder (or pressing Enter in a path field) loads it immediately; there are no separate "Load" buttons. HDF5 dataset / frame-index changes reload automatically.
  - **No accidental scroll changes.** Spin boxes and drop-downs ignore the mouse wheel — values change only by clicking/typing; the wheel scrolls the panel instead.
  - **Readable right-click menus.** pyqtgraph plot context menus use the dark theme.
  - **Dark theme, orange accent** (Dioplas-inspired); off-white text, light input fields.
- 

## 13. Packaging, Deployment & Diagnostics

---

**Packaging.** `midas-gui` is a MIDAS-style package: `pyproject.toml` (BSD-3-Clause), a `tests/` smoke suite, and `release.sh` for cutting versioned releases (see `RELEASING.md`). Once published it can join the `midas-suite` meta-package as an optional `gui` extra (`pip install midas-suite[gui]`).

**Deployment on another system.** Clone the repo, create the conda environment (`environment.yml`), and run `midas-gui`. The MIDAS analysis backends are installed via the `pip:` section of `environment.yml` (editable from a local MIDAS checkout, from a git subdirectory, or from a private index). `midas_gui/_paths.py` is an inert stub in an installed package (no `sys.path` manipulation).

**Crash diagnostics.** On startup the app installs a logging exception hook and `faulthandler`; any uncaught Python exception or native fault is written to `~/midas_gui_error.log` and shown in a dialog. Installing the hook also prevents PyQt5 from hard-aborting on an exception inside a slot. Each tab is built in isolation — a tab that fails becomes an error placeholder instead of taking the window down. If the app ever "pops up and dies" (typically on Windows), send that log file.

---

*For bugs or questions, see the MIDAS GUI repository.*